

Engineering-First Deep Understanding

Panel Introduction

Staff/Principal Product SRE: What follows is not theory. Every scenario is based on real production incidents. Every counterexample represents actual misdiagnoses I've witnessed. This is the knowledge gap between engineers who survive on-call and those who don't.

Distributed Systems Engineer: The focus is predicting system behavior under failure. If you can't model these scenarios mentally before they happen, you'll be reactive and slow during incidents.

Incident Commander: These are the decision points that separate 5-minute mitigations from 3-hour investigations. Understanding the "why" enables fast triage.

1. The Four Golden Signals: Production-Grade Understanding

From OS Engineer Perspective

The golden signals map directly to kernel resource management:

Latency: Time spent in runqueues, I/O wait, network buffers

- Low CPU but high latency → Process blocked (I/O, network, locks)
- High CPU and high latency → Process compute-bound or context-switching

Traffic: System calls per second, file descriptor creation rate

- Sudden traffic drop → Upstream timeout, circuit breaker opened, or client failure
- Traffic spike → DDoS, retry storm, or legitimate growth

Errors: Failed system calls (EAGAIN, ECONNREFUSED, ETIMEDOUT)

- Error rate at syscall level often hidden from application metrics
- Example: Database connection failures logged but not in HTTP error rate

Saturation: Resource limits hit (fd limit, connection pool size, queue depth)

- Can occur at any layer: application, OS, network, database
- Non-linear degradation at saturation boundaries

Production Scenario 1: High Latency, Low Utilization

Symptom:

```
Service: Payment API
P99 Latency: 8000ms (SLO: 500ms)
CPU: 15%
Memory: 30%
Error Rate: 0%
Traffic: 100 req/s (normal)
```

Common Wrong Diagnosis: "System is underutilized, so it should be fast"

Actual Root Cause (one of):

1. Database Connection Leak

```
# Buggy code
def process_payment():
    conn = db.connect()
    result = conn.execute(query)
    return result
    # conn.close() never called
```

- Connection pool exhausted
- New requests wait for available connection
- CPU idle (waiting), but latency high

1. Downstream Service Timeout

```
Payment API → Fraud Detection API (timeout: 30s)
Fraud API degraded (responding in 25s)
Payment API waits 25s per request
P99 dominated by this wait time
```

1. Lock Contention

```
with global_lock:
    process_request() # Slow operation
# All requests serialize on this lock
# CPU idle (waiting for lock)
# Latency = queue depth × processing time
```

Counterexample: When Error Rate is 0% But System is Failing

Scenario:

```
Service: User Profile API
Error Rate: 0% (all 200 OK responses)
Complaints: "Users seeing wrong data"
```

Investigation:

```
# Check cache hit rate
GET /metrics | grep cache
cache_hit_rate = 99.9%

# Check cache TTL
redis-cli TTL user:12345
300 (5 minutes)

# Check database replication lag
SHOW SLAVE STATUS\G
Seconds_Behind_Master: 180 (3 minutes)
```

Root Cause: Cache serving stale data from lagging replica

Why Error Rate is 0%:

- Requests succeed (200 OK)
- Data is wrong, but application doesn't know
- No errors logged

Correct Metrics:

- Data freshness SLI (timestamp of data vs current time)
- Replication lag metric
- Cache hit rate vs accuracy

Production Scenario 2: Low Traffic, High Saturation

Symptom:

```
Service: Image Processor
Traffic: 10 req/s (very low)
Memory: 95% (saturated)
Latency: 1000ms (SLO: 200ms)
```

Wrong Diagnosis: "Need more instances to handle load"

Actual Root Cause: Memory leak

```
# Buggy code
processed_images = [] # Global list

def process_image(image):
    result = expensive_operation(image)
    processed_images.append(result) # Never cleared
    return result

# Over time: processed_images grows indefinitely
# Memory exhausted → GC thrashing → high latency
```

Observable Signals:

```
# Memory growth over time
heap_size_bytes (increasing monotonically)

# GC time increasing
gc_pause_seconds (increasing)

# Requests slow down as heap grows
p99_latency correlated with heap_size
```

Fix:

```
def process_image(image):
    result = expensive_operation(image)
    return result # Don't store in global state
```

From SRE / On-Call Perspective

Triage Decision Tree:

```
High Latency Alert Fires
├─ Check Error Rate
│   └─ If high → Upstream dependency failure (rollback/circuit breaker)
│       └─ If low → Continue investigation
├─ Check Traffic
│   └─ If spiking → Load issue (scale up/rate limit)
```

- | └─ If normal → Resource saturation or dependency
- | └─ Check Resource Saturation
- | └─ CPU high → Compute-bound (scale up)
- | └─ Memory high → Memory leak or cache overload
- | └─ Connections high → Connection pool exhaustion
- | └─ All low → Downstream latency (check dependencies)

From Hiring Manager Perspective

Interview Question 1:

"Your service's P99 latency suddenly increased from 100ms to 5000ms. CPU and memory are both at 30%. What do you investigate?"

Signals I'm Looking For:

- Candidate checks downstream dependencies first
- Mentions connection pools, database, network
- Understands that low resource utilization $\neq$ healthy
- Asks about recent deploys

Red Flags:

- Only focuses on CPU/memory
- Suggests "add more instances" immediately
- Doesn't consider dependencies

Interview Question 2:

"You have 0% error rate but users are complaining. How is this possible?"

Signals I'm Looking For:

- Stale cache / replication lag
- Silent data corruption
- Incorrect success status codes
- Sampling bias (errors not being captured)

Red Flags:

- "If error rate is 0%, there's no problem"
- Blames users
- Doesn't understand metrics can lie

Lab Exercises

Easy: Golden Signals Triage

Setup:

```
# docker-compose.yml
services:
  api:
    image: sample-api
    environment:
      DB_HOST: postgres
  postgres:
    image: postgres:15
  prometheus:
    image: prom/prometheus
  grafana:
    image: grafana/grafana
```

Exercise:

1. Deploy stack
2. Inject each failure mode:
 - Slow database queries
 - Connection pool exhaustion
 - Memory leak
3. For each: Identify which golden signal detects it first
4. Time yourself: Detection → diagnosis → mitigation

Success Criteria:

- Detect within 2 minutes
- Correctly identify root cause within 5 minutes
- Mitigate within 10 minutes

Moderate: Ambiguous Metrics

Setup: Multi-service app (API → Cache → Database)

Inject: Database replication lag (3 minutes behind)

Exercise:

1. All golden signals show "healthy"
2. Users report stale data
3. Design new metrics to detect this
4. Implement metrics
5. Create alert that would have caught it

Success Criteria:

- Metric measures actual problem (data freshness)
- Alert fires before user complaints
- Low false positive rate

Hard: Cascade Failure

Setup: 5-service dependency chain

Inject: Service C timeout increases from 100ms → 2000ms (gradual)

Exercise:

1. Monitor golden signals for all 5 services
2. Observe cascade propagation
3. Identify: Which service's metrics showed problem first?
4. Calculate: How long until blast radius reached Service A?
5. Design: Circuit breaker thresholds to contain failure at Service B

Success Criteria:

- Correctly identify initial failure point
- Measure cascade timing
- Circuit breaker limits blast radius to 2 services (not all 5)

Expert-Level Diagnostic Questions

Question 1:

You have three metrics:

- `request_duration_seconds_sum`
- `request_duration_seconds_count`
- `request_duration_seconds_bucket`

Which one should you use for alerting on high latency? Why?

Expected Answer: Histogram buckets with `histogram_quantile()` to get P99. Sum/count gives average, which hides tail latency. Production failures often manifest in tail latency first.

Question 2:

Your service handles 1000 req/s. Error rate is 1%. You add retry logic (retry once on error). What's the new request rate to your database? What's the risk?

Expected Answer:

- Original: 1000 req/s, 10 req/s fail
- With retry: $1000 + 10 = 1010$ req/s (+1% load)
- If all downstream services do this: Multiplicative amplification
- Risk: Retry storm during high error rates

Question 3:

Service A calls Service B. B's P99 latency is 500ms. You set A's timeout for calling B to 600ms. Good or bad? Why?

Expected Answer:

Bad. P99 means 1% of requests take > 500ms. If timeout is only 600ms, 1% of requests will timeout. This triggers retries, alerts, etc. Better: Set timeout at P99.9 or P99.99 to cover normal latency variation.

2. SLIs/SLOs/Error Budgets: Negotiation and Enforcement

From Product SRE Perspective

SLIs are a contract between you and your users. Not between you and your manager. If your SLI doesn't capture user pain, it's worthless.

Production Scenario 3: Misleading SLO Compliance

Setting:

```
Service: Video Streaming API
SLO: 99.9% of requests return within 2000ms
Reality: 99.95% compliance (exceeding SLO!)
User Complaints: "Videos won't load"
```

Investigation:

```
-- Check latency by endpoint
SELECT endpoint, p99_latency
FROM metrics
WHERE service = 'video'

Results:
/api/search      → 50ms
/api/metadata    → 80ms
/api/stream/start → 15000ms (!)
```

Root Cause: SLO measured aggregate latency across all endpoints

Problem:

- `/api/stream/start` is 0.05% of traffic
- But it's the critical user path

- High latency on this endpoint ruins experience
- Aggregate SLO hides this

Correct SLO Design:

```
SLO 1: 99.9% of /api/stream/start requests < 500ms
SLO 2: 99.9% of /api/search requests < 200ms
SLO 3: 99.9% of /api/metadata requests < 500ms

Critical: Separate SLOs for critical path endpoints
```

From Infrastructure Architect Perspective

Multi-Service SLO Composition

```
User Request Path:
Frontend → API Gateway → Auth → Product → Payment
  99.9%       99.9%       99.9%   99.9%       99.9%

Composite Reliability:  $0.999^5 = 0.995 = 99.5\%$ 
```

Critical Insight: User-facing SLO degrades multiplicatively

Implication:

- If you promise 99.9% to users
- And you have 5-service chain
- Each service needs 99.98% SLO (not 99.9%)

Real Calculation:

```
target_user_slo = 0.999
num_services = 5
required_per_service = target_user_slo ** (1 / num_services)
# = 0.9998 = 99.98%
```

Production Scenario 4: Error Budget as Negotiation Tool

Setting: Q4 Planning, Product wants to launch new feature

Product Manager: "We need to launch Black Friday feature by Nov 15"

SRE: "Current error budget status:"

Month: November (30 days)
SLO: 99.9% uptime
Error Budget: 43.2 minutes/month
Used So Far: 38 minutes (15 days in)
Remaining: 5.2 minutes (for next 15 days)

SRE Analysis:

- New feature has 10% risk of incident
- Average incident duration: 45 minutes
- Expected cost: $0.1 \times 45 = 4.5$ minutes of downtime
- Would consume 86% of remaining budget

Negotiation Options:

Option 1: Launch with mitigation

- Feature flag for fast rollback
- Staged rollout (1% → 10% → 100%)
- 24/7 on-call coverage during launch
- Accept risk

Option 2: Defer to December

- Fresh error budget (43.2 minutes)
- More time for testing
- Safer launch

Option 3: Reduce SLO for November

- Temporarily relax to 99.5% (2.5 hours downtime allowed)
- Communicate to stakeholders
- Launch feature

Product Decision: Option 1 with feature flag

Outcome: Launch successful, 2 minutes of downtime (within budget)

From Hiring Manager Perspective

Interview Question 3:

"Your service has 99.9% availability SLO. It's November 25th and you've used 90% of your error budget. Product wants to launch a feature on November 28th. What do you do?"

Signals I'm Looking For:

- Quantifies risk (remaining budget vs expected cost)
- Proposes mitigation (feature flags, staged rollout)

- Involves stakeholders in decision
- Understands error budget is a negotiation tool

Red Flags:

- "Just launch it" (ignores SLO)
- "Say no" (doesn't propose alternatives)
- "Ask my manager" (doesn't own the decision)

Counterexample: When 99.99% SLO is Wrong

Scenario: Internal admin tool for support team

Proposed SLO: 99.99% (52 minutes downtime/year)

Reality Check:

- 10 users (support team)
- Used during business hours only (40 hours/week)
- Downtime outside business hours = no impact
- Cost to achieve 99.99%: \$50k/year (HA setup, on-call)
- Business value: Low

Correct SLO: 99% during business hours (9am-5pm weekday)

- Allows 3.6 hours downtime/year during business hours
- Much cheaper to operate
- Matches actual user expectations

Key Insight: Higher SLO $\neq$ better. Right SLO = matches user needs and cost tradeoffs.

Lab Exercises

Easy: SLI/SLO Definition

Setup: Simple HTTP API

Exercise:

1. Identify user-critical endpoints
2. For each endpoint:
 - Define SLI (latency, availability, or both)
 - Choose SLO threshold (99%, 99.9%, 99.99%?)
 - Justify why (cost vs benefit)
3. Implement measurement in Prometheus
4. Calculate error budget

Success Criteria:

- SLI captures user experience
- SLO threshold justified with reasoning
- Measurement works (can query current compliance)

Moderate: Error Budget Exhaustion

Setup: Service with 99.9% SLO, currently at 99.85% (budget 50% exhausted)

Exercise:

1. Calculate days remaining in measurement window
2. Calculate remaining error budget minutes
3. Model risk: If you have 1 incident/week averaging 20 minutes, will you exhaust budget?
4. Decide: Freeze launches or continue? Why?

Success Criteria:

- Math correct
- Decision justified with quantitative reasoning
- Proposes mitigation if continuing

Hard: Multi-Service SLO Composition

Setup: 5-service dependency chain, each with independent SLO

Exercise:

1. Measure actual user-facing reliability (end-to-end)
2. Calculate composite SLO
3. Identify: Which service is the weakest link?
4. Propose: How to improve user-facing SLO to 99.9%?
 - Option A: Improve weakest service
 - Option B: Add redundancy
 - Calculate cost/benefit of each

Success Criteria:

- Composite calculation correct
- Identifies bottleneck service
- Proposes justified improvement plan

Expert-Level Diagnostic Questions**Question 4:**

Your API has two endpoints: `/read` (99% of traffic) and `/write` (1% of traffic). You have one SLO: "99.9% of requests succeed." Is this good? Why or why not?

Expected Answer:

Bad. Writes are probably more critical (data loss risk). If writes fail 10% of the time but reads succeed 99.9%, aggregate SLO is ~99.8% and looks good. But users can't write. Need separate SLOs for read and write operations.

Question 5:

You have 99.9% SLO and 10% error budget remaining with 5 days left in month. Product wants to deploy. The deploy has 5% chance of causing a 30-minute outage. Do you approve? Show your math.

Expected Answer:

- Error budget remaining: $0.1 \times 43.2 \text{ min} = 4.32 \text{ min}$
 - Expected cost: $0.05 \times 30 \text{ min} = 1.5 \text{ min}$
 - Remaining after deploy: $4.32 - 1.5 = 2.82 \text{ min}$ (65% remaining)
 - Decision: Yes, but require fast rollback mechanism and monitoring
 - If incident happens: Still have buffer
-

3. Incident Response: MTTR Optimization

From Incident Commander Perspective

The first 5 minutes determine MTTR. Fast, correct triage turns 3-hour incidents into 15-minute incidents.

Production Scenario 5: Premature Root Cause Analysis

Incident Start: 14:32 - Payment API returning 500 errors

Timeline (What Happened):

```
14:32: Alert fires - Payment API error rate 25%
14:33: Engineer A starts investigating logs
14:38: Engineer A identifies suspicious DB query in logs
14:43: Engineer A optimizes query, creates PR
14:58: PR merged, deployed
15:05: Payment API still returning 500s (no change)
15:08: Engineer B joins, checks recent deploys
15:09: Identifies deploy at 14:30 (2 min before incident)
15:10: Rollback initiated
15:12: Service recovered
```

MTTR: 40 minutes (14:32 → 15:12)

Timeline (What Should Have Happened):

```
14:32: Alert fires
14:33: Check recent deploys → Deploy at 14:30
14:34: Rollback initiated
14:36: Service recovered
```

MTTR: 4 minutes

Root Cause of Long MTTR: Engineer A debugged before mitigating

Principle: Mitigate first, debug later

Production Scenario 6: Incorrect Severity Classification

Incident: E-commerce site, Checkout API slow (5s latency, SLO: 500ms)

Initial Classification: SEV3 (Low priority)

- Reasoning: "Error rate is only 5%, most requests succeeding"

Reality:

- Checkout is revenue-critical
- 5s latency = users abandon cart
- Revenue impact: \$50k/hour

Correct Classification: SEV1 (Critical)

Severity Matrix:

Impact	User Facing	Revenue Impact	Severity
Total outage	Yes	High	SEV1
Partial degradation	Yes	High	SEV1
Partial degradation	Yes	Low	SEV2
Total outage	No (internal)	Low	SEV2
Degradation	No	Low	SEV3

Result of Misclassification:

- SEV3 → Response time: 1 hour
- SEV1 → Response time: 5 minutes
- Cost: 55 minutes of revenue loss (\$45k)

From On-Call Engineer Perspective

Mitigation Playbook (Ordered by Speed):

1. Rollback (2 minutes)
 - Recent deploy causing issues?
 - Instant fix, no investigation needed
2. Feature Flag Disable (1 minute)
 - New feature causing issues?
 - Disable flag, service recovers
3. Scale Up (5 minutes)
 - Load spike?
 - Auto-scaling slow to react?
4. Circuit Breaker / Rate Limit (2 minutes)
 - Downstream service failing?
 - Open circuit breaker to prevent cascade
5. Service Restart (3 minutes)
 - Memory leak?
 - Connection pool exhaustion?
 - Restart resets state
6. Database Failover (10 minutes)
 - Primary database down?
 - Failover to replica
7. Traffic Reroute (15 minutes)
 - Region down?
 - Route traffic to healthy region
8. Debug and Fix (30+ minutes)
 - Unknown root cause
 - Last resort

Key Principle: Prefer fast, ugly mitigations over slow, proper fixes

Production Scenario 7: Mitigation Causes Cascade

Incident: Auth Service overloaded (1000 req/s, capacity: 800 req/s)

Mitigation Attempt: Scale from 10 instances → 100 instances

Result:

```
15:00: Scale-up initiated
15:03: 100 Auth instances running
15:04: Database connection pool exhausted (100 instances × 10 conn/instance = 1000 connections)
15:04: Database refusing connections
15:05: All Auth instances failing (can't connect to DB)
15:05: Complete Auth outage
```

Correct Mitigation:

```
Option 1: Rate limit at load balancer (immediate)
- Reject excess traffic
- Protect downstream database

Option 2: Scale gradually
- 10 → 20 → 40 (monitor DB connection pool)
- Increase DB connection pool limit in parallel

Option 3: Cache aggressively
- Increase cache hit rate
- Reduce DB load
```

Lesson: Mitigation can make things worse. Consider downstream impact.

From Hiring Manager Perspective

Interview Question 6:

"You deploy a new feature at 2pm. At 2:05pm, error rate spikes to 10%. What do you do in the first 5 minutes?"

Correct Answer:

1. Rollback immediately (don't investigate yet)
2. Verify service recovers
3. THEN investigate why feature caused errors

Red Flags:

- "I'd check the logs first"

- "I'd try to fix the bug"
- Doesn't mention rollback

Interview Question 7:

"Your service depends on Database A. Database A goes down. What do you do?"

Correct Answer:

- Immediate: Open circuit breaker to prevent cascade
- Short-term: Serve cached/stale data if possible
- Medium-term: Failover to Database B (replica)
- Don't: Keep retrying and taking down your service too

Red Flags:

- "Wait for database team to fix it"
- Doesn't mention circuit breaker or graceful degradation

Lab Exercises

Easy: Rollback Race

Setup: Service with feature flag system and deployment pipeline

Exercise:

1. Deploy change that introduces errors
2. Detect error (should be immediate via metrics)
3. Timed challenge:
 - Option A: Rollback deployment
 - Option B: Disable feature flag
 - Which is faster? Measure both.

Success Criteria:

- Detection < 1 minute
- Mitigation < 3 minutes
- Document which mitigation was faster

Moderate: Severity Classification

Setup: You're given 10 incident scenarios

Exercise: For each, classify SEV1/SEV2/SEV3 and justify

Examples:

1. Checkout API: 100% error rate
2. Internal admin tool: Slow (5s latency)
3. Recommendation engine: Returning empty results

4. Database replica: 2-hour replication lag
5. CDN edge location: Offline (1 of 50 locations)

Success Criteria:

- Correct severity for 8/10 scenarios
- Justification includes user impact and revenue impact

Hard: Cascade Prevention

Setup: 5-service chain ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$), each with connection pools

Exercise:

1. Inject: Service E latency increases to 10s
2. Observe: Measure how long until Service A is affected
3. Mitigate: Implement circuit breaker at B (between A and B)
4. Re-test: Prove Service A remains healthy when E fails

Success Criteria:

- Measure cascade timing (baseline)
- Circuit breaker contains failure (Service A unaffected)
- Document: At what threshold should circuit open?

Expert-Level Diagnostic Questions

Question 6:

During incident, you have two options:

- Option A: Rollback deploy (2 min to recover)
- Option B: Fix bug in code (20 min to recover)

Error budget status: 80% exhausted. Which do you choose? Why?

Expected Answer:

Rollback. Mitigation speed matters more than proper fix. Error budget nearly exhausted, can't afford 20 minutes of downtime. Rollback, then fix bug properly offline.

Question 7:

You scale service from 10 to 100 instances. Service starts failing. Why might this happen? Name 3 reasons.

Expected Answer:

1. Downstream connection pool exhaustion (DB can't handle 100x connections)
2. Rate limits hit (API quota exceeded with 10x traffic)
3. Broadcast storms (if instances communicate via broadcast)
4. Coordinated cache miss (100 instances all request same data simultaneously)

[Continue with sections 4-10 in similar detail...]

Due to length constraints, I'll create separate files for remaining sections.